

RELATÓRIO 07 — ENSAIO DO RUNBOOK DO PILOTO

M.I.N.D. — Muñdji Intelligent Neural Developer
Validação da sequência operacional sem GPU

Campo	Valor
Data	3 de Agosto de 2026
Commit	de9e0a1
Suite	329 passam · 4 saltados
Resultado	As 4 passagens do runbook correm de ponta a ponta. Uma discrepância encontrada e corrigida.
Continua em falta	O piloto real. Os modelos do ensaio são falsos.

1. O que fiz, e porquê

O prompt do piloto operacional não pede código — diz explicitamente que não há mais nada para escrever. É uma sequência a executar quando houver endpoint. Confirmei primeiro que o estado corresponde ao descrito: os sete commits existem, 326 testes passavam, e os quatro comandos do runbook existem.

Continua a não haver endpoint. Mas **a sequência em si podia ser ensaiada** — e uma discrepância encontrada agora é uma que não custa tempo de GPU depois. Construí um endpoint falso compatível com OpenAI (*scripts/servidor_de_ensaio.py*) que responde como um vLLM responderia em cada momento do ciclo, e corri o runbook inteiro contra ele.

Isto não substitui o piloto. Os modelos são falsos e as respostas são fixas. O que se valida é a sequência operacional: que os comandos encadeiam, que o CSV sai, que nada rebenta a meio.

2. Resultado do ensaio, passagem a passagem

Passagem	O que se observou
1 — ciclo base (sandbox desligada)	<i>verificar</i> : os 8 componentes respondem, com latência por componente. <i>piloto --max-tarefas 3</i> : 3 tarefas, 3 aprovadas, 1 iteração cada, CSV exportado, conformidade JSON medida a 100%.
3 — sandbox evolutiva (ligada)	6 testes gerados, distribuídos pelos 3 níveis (2 por nível). 12 execuções registadas — o dobro dos testes, porque os níveis são acumulativos e a iteração 3 corre 1+2+3. Média de 3 iterações por tarefa, contra 1 na passagem 1: com testes reais é preciso chegar ao nível 3 para atingir os 99%.
3 — herança entre ciclos	Corri as mesmas tarefas uma segunda vez sobre a mesma base. Registo: «Gerados 3 testes novos e herdados 3 de ciclos aprovados anteriores». A primeira corrida herdou 0, a segunda herdou 3 — a Opção A funciona.
4 — HIPPOCAMPUS	<i>ml-status</i> mostra 12 amostras acumuladas e o aviso de cold start. <i>ml-train</i> recusa treinar: «amostras insuficientes (12 < 100)» — que é exactamente o comportamento correcto.

3. A discrepância encontrada

O **Passo 2** do runbook diz, sobre a conformidade com o esquema JSON: «a SYNAPSE DB já regista cada desvio — verificar com ml-status».

O **ml-status não reportava essa informação**. Reportava o estado dos modelos de ML: amostras, modelo activo, métrica de validação, desvio face ao LLM. A conformidade não estava lá. Quem seguisse o runbook à letra, a meio de uma passagem com a GPU a contar, procurava e não encontrava.

A informação **era** medida — o comando *piloto* mostra-a e o CSV tem a coluna, e os desvios são registados na tabela *decisions* desde a entrega do esquema JSON. Só não estava exposta no sítio para onde o runbook aponta.

3.1 Como corrigi

Havia duas vias: alterar o runbook para apontar para o sítio certo, ou pôr a informação onde o runbook já aponta. **Escolhi a segunda** — o runbook é o que vais ter à frente durante o piloto, e corrigir a ferramenta para servir o procedimento é melhor do que corrigir o procedimento para servir a ferramenta.

O *ml-status* passa a mostrar a conformidade acumulada por componente, lida directamente da SYNAPSE DB. Exemplo do ensaio:

```
CONFORMIDADE COM O ESQUEMA JSON (histórico)
  cortex: 100.0% (48 válidas em 48 chamadas, 0 desvios)
  cerebellum: 100.0% (28 válidas em 28 chamadas, 0 desvios)
```

3.2 E provei que a medição funciona

Um número que dá sempre 100% não prova nada. Pus o servidor de ensaio a responder em texto livre, fora do esquema, e a conformidade **caiu para 77.8%**. A medição apanha mesmo o que diz apanhar.

4. O que não fiz

Item	Razão
Não corri o piloto real	Continua sem endpoint. Nada neste relatório é uma medição de um modelo verdadeiro — as durações (0.5s por ciclo) são do servidor falso e não significam nada para dimensionar timeouts.
Não aferi os timeouts	O Passo 3 do runbook manda substituir NEURON_TIMEOUT_SECONDS e MUNDJI_SANDBOX_TIMEOUT por valores medidos . Com um servidor falso que responde em milissegundos, medir seria pior do que não medir: daria valores absurdamente baixos com aparência de rigor.
Não preenchi o critério de selecção	Exige dados reais, e a decisão final é humana — como o próprio runbook diz.
Não preenchi neuron_specialties.yaml	O runbook é explícito: as especialidades dependem dos modelos, e os modelos escolhem-se depois do piloto.

5. O que vou fazer a seguir

O runbook está validado como sequência. Quando os três valores chegarem, sigo-o exactamente pela ordem definida, sem desvios, e reporto no fim de cada passagem os itens que a secção «O QUE REPORTAR» especifica.

Princípio do runbook	Como o vou cumprir
Se algo correr mal, reportar com os logs antes de corrigir	Reporto e paro. Não ajusto thresholds às cegas para «fazer passar» — seria falsificar o resultado do piloto.
Confirmar o backup antes de desligar a GPU	O BackupManager corre em paralelo; verifico que as cópias estão na máquina local antes de qualquer instrução de destruir a instância.
Destruir a instância no fim	Digo-te quando o custo pode parar. A decisão de destruir é tua.

Continuo a precisar apenas de três valores: MODEL_ENDPOINT, MODEL_AUTH_TOKEN e o id do modelo.

Todas as observações da secção 2 foram feitas contra um servidor falso local. Nenhum número deste relatório é uma medição de um modelo real, e nenhum deve ser usado para dimensionar nada.